PhoSem
Photonic.Semiconductor

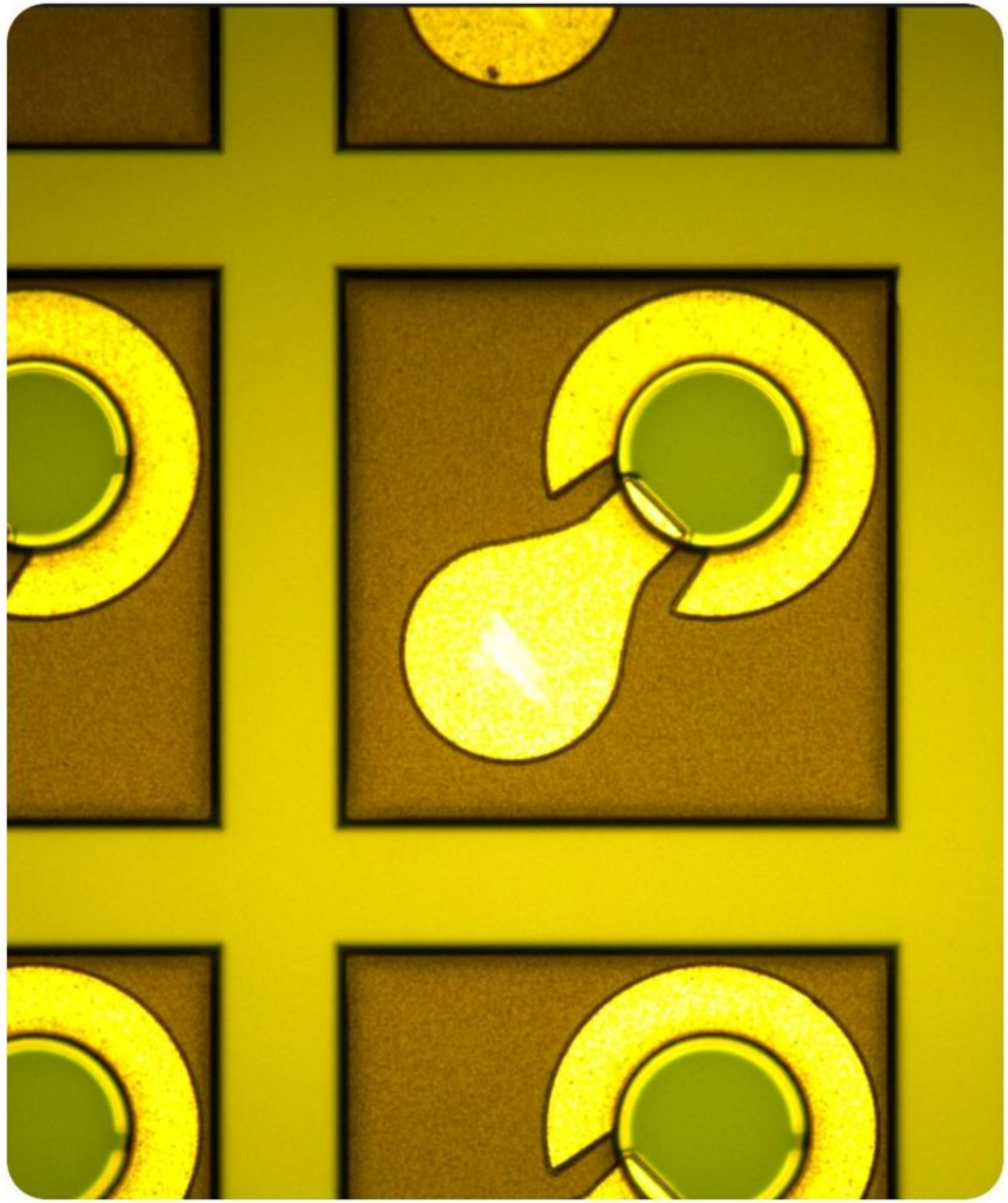
Project Retrospective

PIN 다이오드 불량 검출 AI

전체 프로젝트 회고 및 발표

CNN 분류 → 데이터 분석 → PatchCore + MIL

염세현 | 2026. 10. 15



Contents

01

프로젝트 시작

과제와 역할 분담

02

첫 시도

250장으로 시작

03

데이터와 피드백

불균형, 멘토링

04

CNN 개발의 정석

EDA와 새 기법

05

새로운 모델

PatchCore + MIL

06

회고와 배운 점

이상과 현실, 다음 과제

01

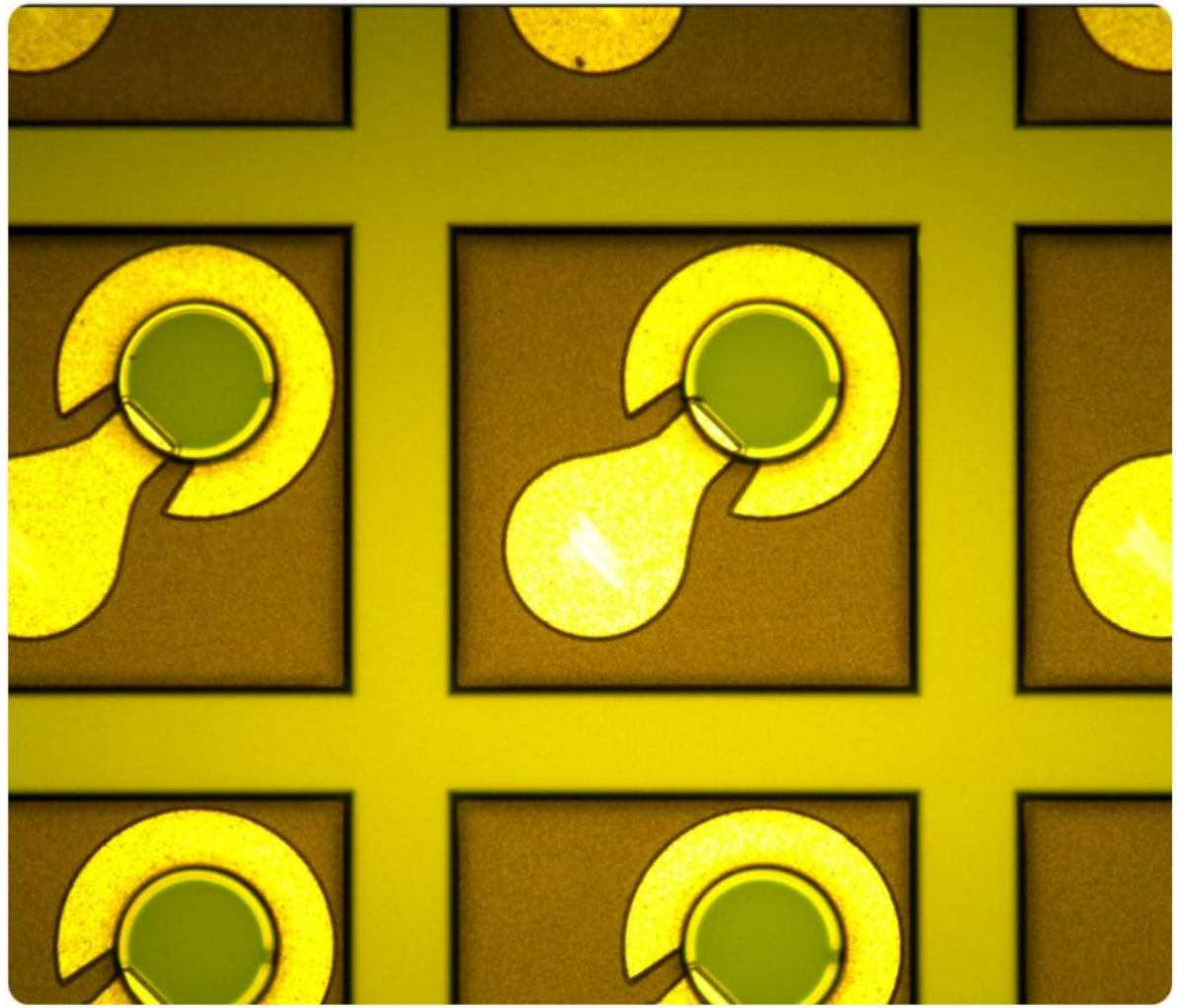
프로젝트 시작

과제 정의, 팀 구성, 개발 순서

사진으로 찾는 PIN 다이오드의 불량

목표는 다이오드 사진을 보고 불량 여부를 판단하고, 불량이라면 그 사유까지 함께 판단하는 모델이었습니다.

다이오드 자체는 단순한 형태라 얕은 신경망으로도 충분해 보였지만, 실제 Plate 위의 다이오드는 정말 다양한 이유로 문제를 일으켰습니다.



촬영 원본 — 한 장에 3×3 이상의 다이오드

세 파트로 나눈 파이프라인

Step 1 · Crop

이미지 자르기

AI 없이 위치 기반으로 다이오드 한 개씩 잘라내는 앱, 전체 앱 통합

김성민 · 3학년

→

Step 2 · Judge

양품 / 불량 판정

PatchCore 이상 탐지로 새 다이오드의 불량 여부 판별

신동호 · 2학년

→

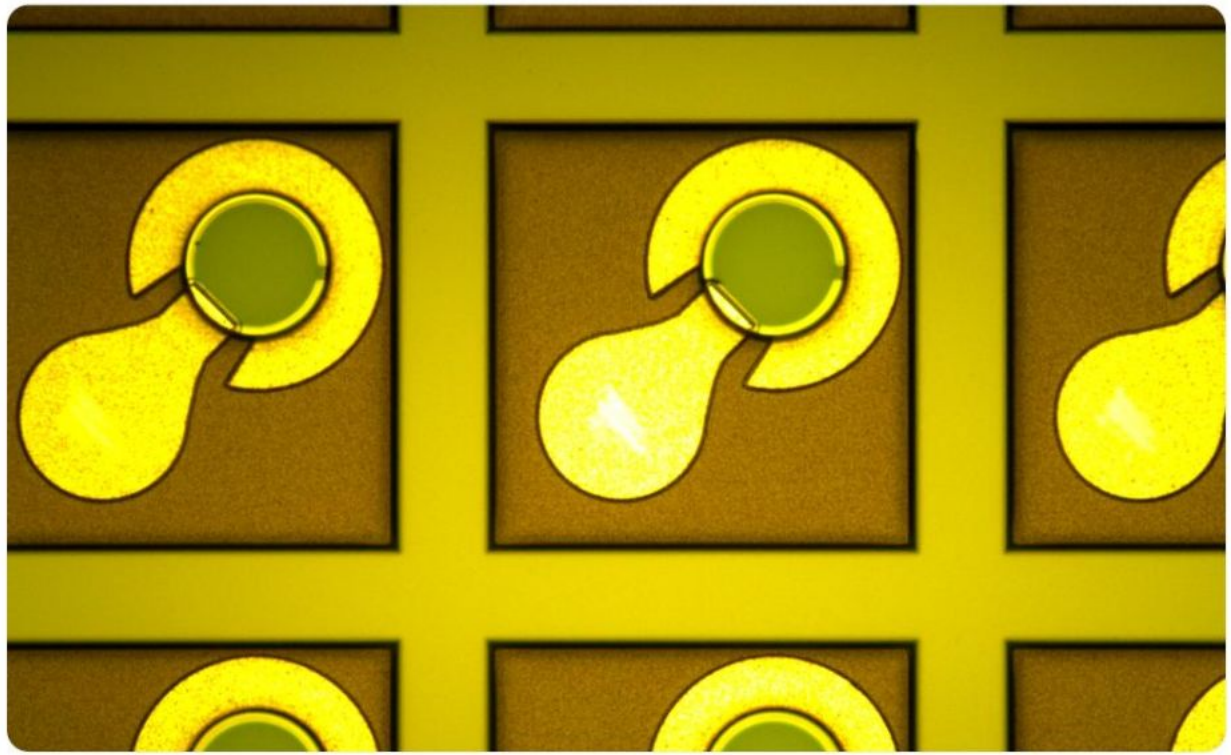
Step 3 · Classify

불량 원인 분류

불량으로 걸러진 이미지를 정해진 오류 클래스로 나눔

염세현 · 2학년 (발표자)

Step 1. 이미지 자르기



Crop 이전 원본 데이터

→



Crop 이후의 전처리 데이터

Step 2. 양품 / 불량 판정



입력: Crop된 다이오드 한 개

PatchCore · 이상 탐지

정상 다이오드와 비교

정상 다이오드의 특징을 모아 둔 기억(Bank)과 새 다이오드를 비교해, 많이 다르면 불량으로 판정합니다.

출력 A

양품

공정 그대로 통과

출력 B

불량

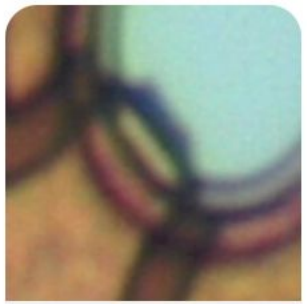
Step 3 원인 분류로 전달

Step 3. 불량 원인 분류

입력

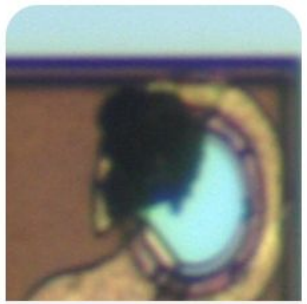
불량 판정된 다이오드

Step 2에서 넘어온 이미지



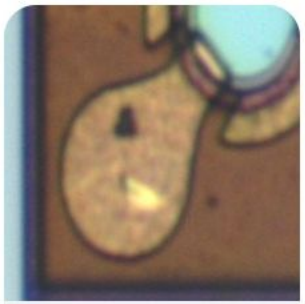
C

Connect
접합



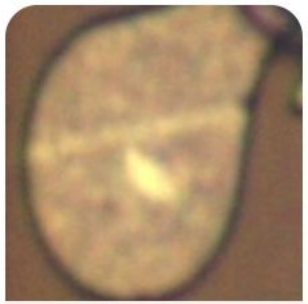
D

Destroy
파손



P

Particle
이물질



S

Scratch
스크래치



B

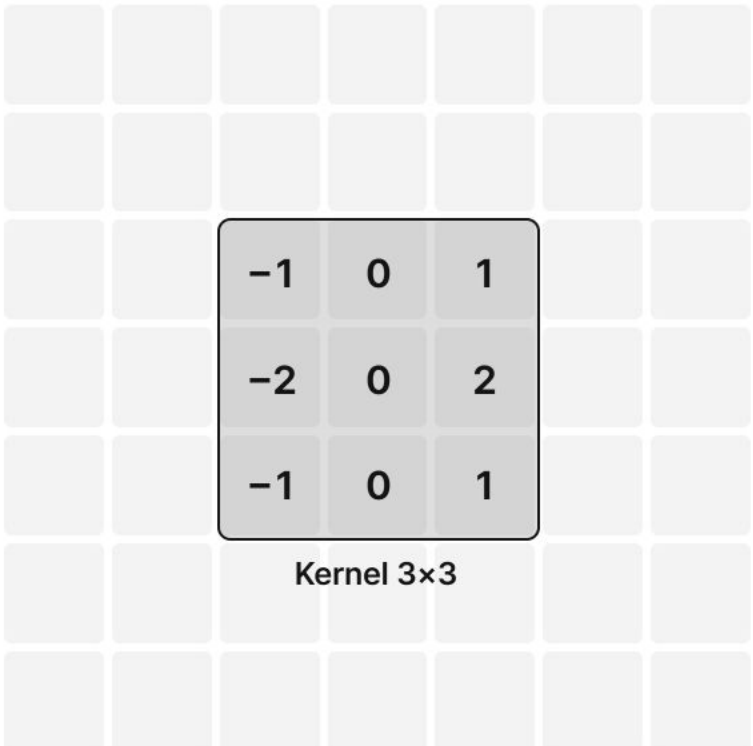
Break
훼손

02

첫 시도

Basic CNN, ResNet

CNN이 이미지를 보는 방식



Convolutional Neural Network

합성곱 신경망. 여러 개의 Conv Layer를 쌓아 만듭니다.

Kernel

작은 틀(프레임)이 이미지 위를 슬라이딩 윈도우 방식으로 움직이며 특징을 찾습니다. 커널과 모양이 비슷한 부분일수록 강하게 반응합니다.

Feature map

커널이 찾아낸 특징이 모인 지도. 층이 깊어질수록 더 복잡한 모양을 알아봅니다.

커널이 Feature map을 만드는 과정



$$(8\times0 + 8\times0 + 7\times1) + (8\times(-1) + 8\times1 + 6\times(-1)) + (8\times1 + 5\times(-1) + 5\times0) = \mathbf{4}$$

커널과 겹친 9칸을 같은 자리끼리 곱해 모두 더하면 숫자 하나가 나옵니다. 커널을 한 칸씩 옆으로 옮기며(슬라이딩 윈도우) 반복하면 Feature map이 채워집니다.

층을 쌓아 점점 큰 특징을 보는 CNN

입력



다이오드 사진

픽셀 숫자의 묶음

얕은 층

선 · 모서리

단순한 무늬를 찾음

중간 층

모양 · 질감

긁힘, 점, 경계

깊은 층

불량의 단서

이물질, 파손 형태

출력

C	**0.72**
P	0.15
D	0.08
S	0.05

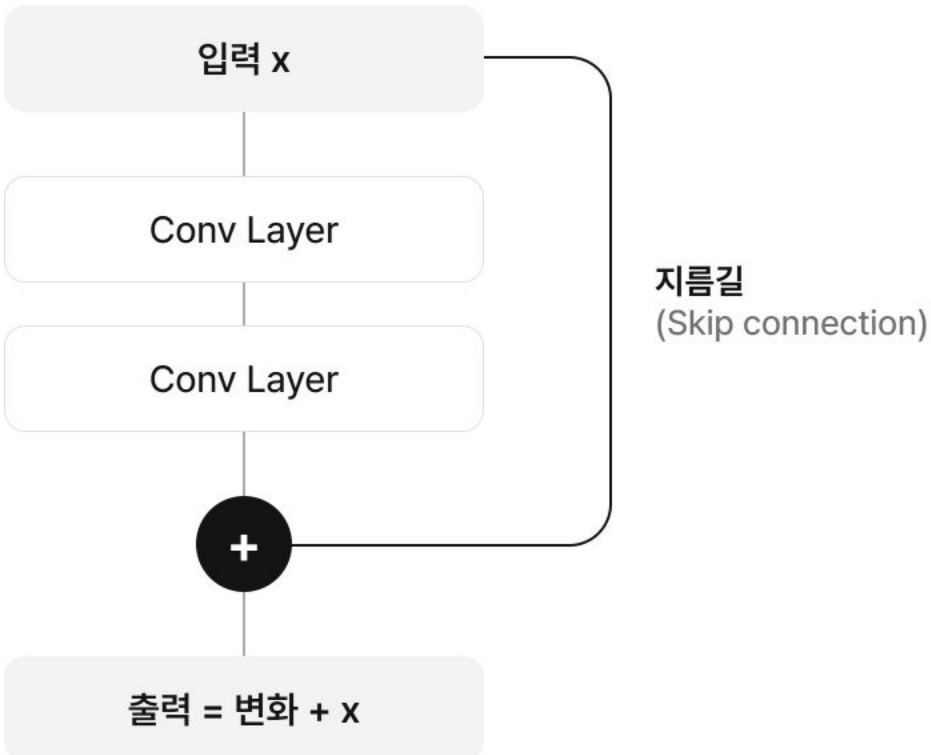
클래스별 확률

가장 높은 것을 정답으로

층을 지날수록 그림의 크기는 작아지고, 대신 '무엇이 있는지'에 대한 정보(채널 수)가 많아집니다. 오른쪽 확률 값은 이해를 돕기 위한 예시입니다.

• 용어 설명

ResNet 모델



문제

층을 깊게 쌓을수록 오히려 학습이 잘 안 되는 현상이 있었습니다.

해결

입력을 몇 층 건너뛰어 출력에 그대로 더해 줍니다. 각 층은 전체를 새로 배우는 대신 '달라진 부분'만 배우면 됩니다.

이 프로젝트에서

레퍼런스가 가장 많은 CNN이라 첫 모델로 골랐습니다. 다만 단순한 다이오드 이미지에는 필요 이상으로 큰 구조였습니다.

첫 모델: ResNet, 데이터 250장

250장

처음 받은 학습 데이터

7:2:1

Train 175 · Val 50 · Test 25 분할

16·32·64

층별 채널 수

98,178

파라미터 개수

당시의 선택

불량을 정상으로 놓치지 않도록 **Threshold**를 정하고, Train 데이터로 **Bank**를 만들어 사용했습니다. Threshold, 아키텍처, 파라미터, 하이퍼파라미터를 배우면서 동시에 개발했습니다.

지금 돌아보면

복잡하지 않은 다이오드를 분류하는 데 ResNet + 최대 64채널 + 98,178개 파라미터는 **꽤 무거운 모델**이었습니다. 첫 모델 개발에서는 파라미터의 중요성을 알지 못했습니다.

Train, Validation, Test



과적합 (Overfitting)

문제집 답을 외워 버린 상태. 본 적 있는 문제는 잘 맞히지만, 처음 보는 문제에서는 무너집니다. 데이터가 적을수록 쉽게 생깁니다.

Threshold와 Recall

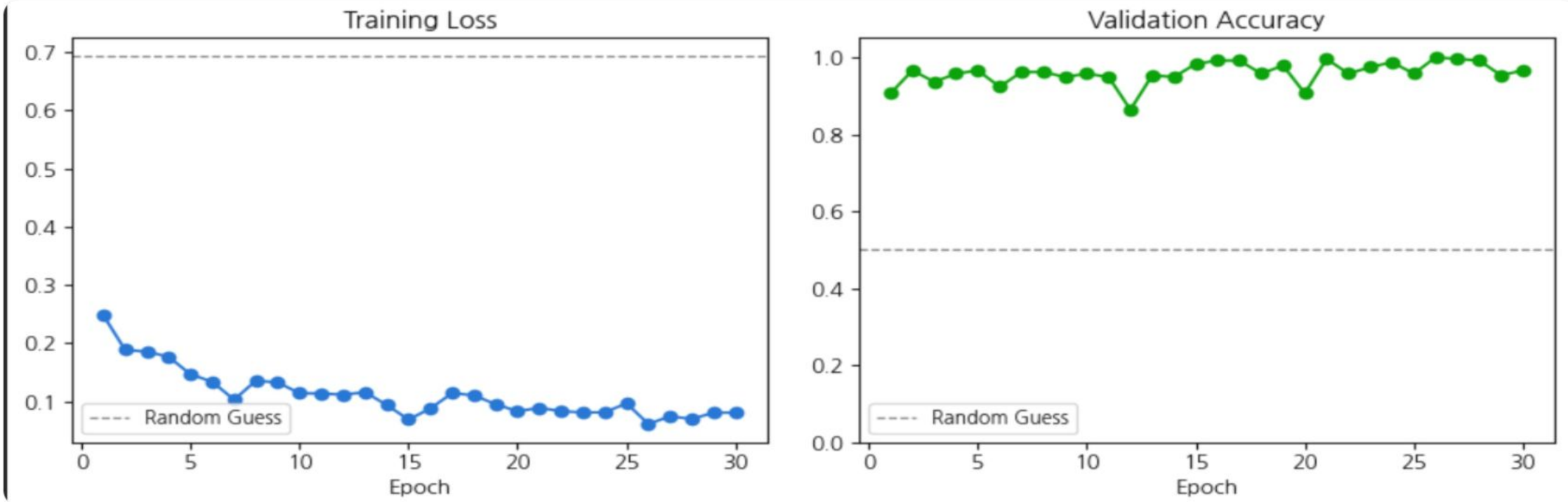
모델이 내놓은 불량 점수로 양품과 불량을 나누는 판단 기준점이 Threshold입니다. Recall은 실제 불량 중 몇 개를 잡아냈는지의 비율입니다.

AP (Average Precision)

모델이 불량이라고 한 것 중 실제 불량의 비율(Precision)을 Threshold 전체에 걸쳐 평균 낸 값. 1에 가까울수록 좋습니다.

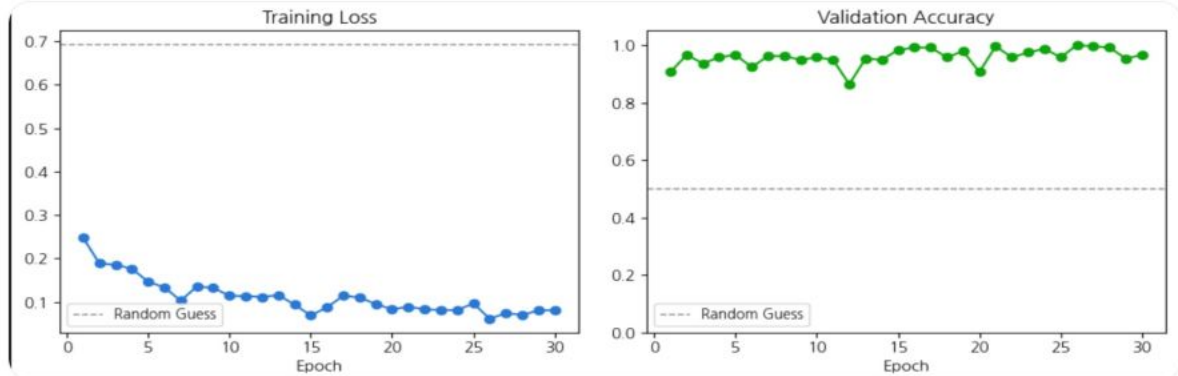
과적합 (Overfitting)

78% 검증 정확도, 30 epoch



Training loss는 꾸준히 내려가지만, Val 정확도는 학습 초반부터 거의 변하지 않습니다. 데이터가 적어 모델이 학습 데이터를 외워 버린 것으로, 처음 보는 데이터에는 약한 전형적인 과적합 상태입니다.

그래프 읽는 법



가로축

Epoch

학습 데이터 전체를 한 번 끝까지 보는 단위. 30 epoch는 같은 데이터로 30번 반복하여 AI가 학습했다는 뜻입니다.

왼쪽 그래프

Training Loss

학습 데이터에서 모델이 틀린 정도로, 낮을수록 학습한 문제를 잘 맞힌다는 것을 의미합니다.

오른쪽 그래프

Validation Accuracy

학습에 쓰지 않은 검증 데이터에서 맞힌 비율. 처음 보는 문제를 얼마나 잘 푸는지 보여 줍니다.

	threshold	불량_recall	놓친_불량	과검_양품
0	0.05	1.000	0	24
1	0.10	1.000	0	24
2	0.15	1.000	0	24
3	0.20	1.000	0	24
4	0.25	1.000	0	24
5	0.30	1.000	0	22
6	0.35	1.000	0	19
7	0.40	1.000	0	18
8	0.45	0.962	1	10
9	0.50	0.808	5	6

Threshold별 불량 Recall · 놓친 불량 · 과검 양품

Threshold를 바꿔도 1로 고정된 Recall

Threshold 양품과 불량을 나누는 판단 기준점

Recall 실제 불량 중 모델이 잡아낸 비율 (1 = 하나도 놓치지 않음)

0.05부터 0.40까지 놓친 불량은 0개였지만, 그 대가로 양품 18~24개를 불량으로 과검했습니다.

숫자는 좋아 보여도 모델이 구분을 배운 결과는 아니었습니다. 원인은 데이터 수량 부족으로 판단했습니다.

03

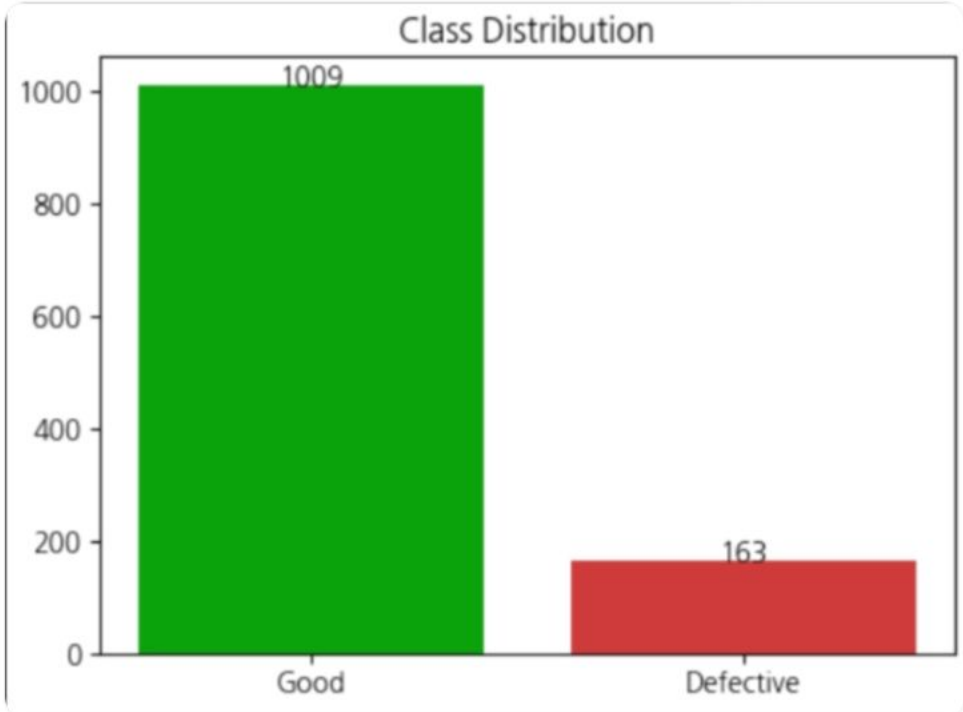
데이터와 피드백

새로 보급된 데이터와 문제

6 : 1

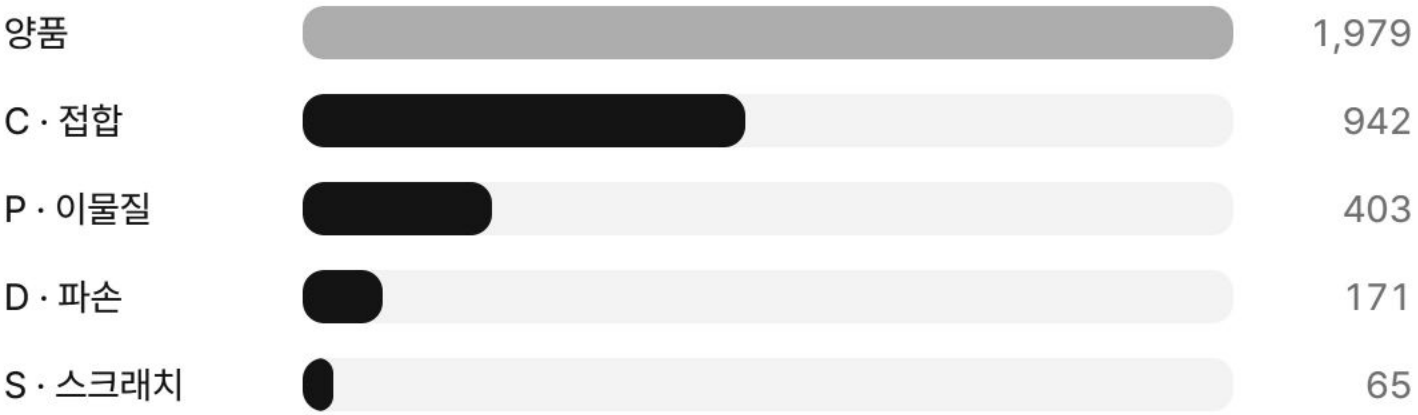
양품 **불량**

실제 다이오드를 촬영해 보내주다 보니 정상 데이터가 불량보다 훨씬 많았고, 이 편향이 다시 과적합을 불렀습니다.



Class Distribution — Good 1,009 · Defective 163

데이터를 더 받아도 여전한 차이



학습 대상 구성 — 지표별 다이오드 수 (중복 포함, B 제외 후 3,266장)

추가 수령 후 (B 제외)

1,172 → **3,266**장

양품 : 스크래치

30 : 1

전체 수는 늘었지만, 원인별로 나누면
가장 적은 클래스는 여전히
65장이었습니다.

상황

데이터를 더 받아도 결과가 나아지지 않았다

불균형은 그대로였고, **과적합**도 계속됐습니다.

질문

문제가 데이터만이 아니라면?

모델 설계 자체를 점검받기 위해 1기 선배님께 멘토링을 요청했습니다.

멘토링에서 들은 세 가지 문제

문제 1

모델이 너무 크다

간단한 이미지 분류에 ResNet 같은 큰 모델을 쓸 필요가 없다.

문제 2

파라미터가 너무 많다

ResNet에서 Feature vector 수까지 크게 잡으니 파라미터가 불필요하게 많다.

문제 3

데이터 품질이 낮다

학습에 쓰는 데이터 자체의 품질이 너무 낮다.

Update

문제 1 · 2 →

Feature vector 수 축소

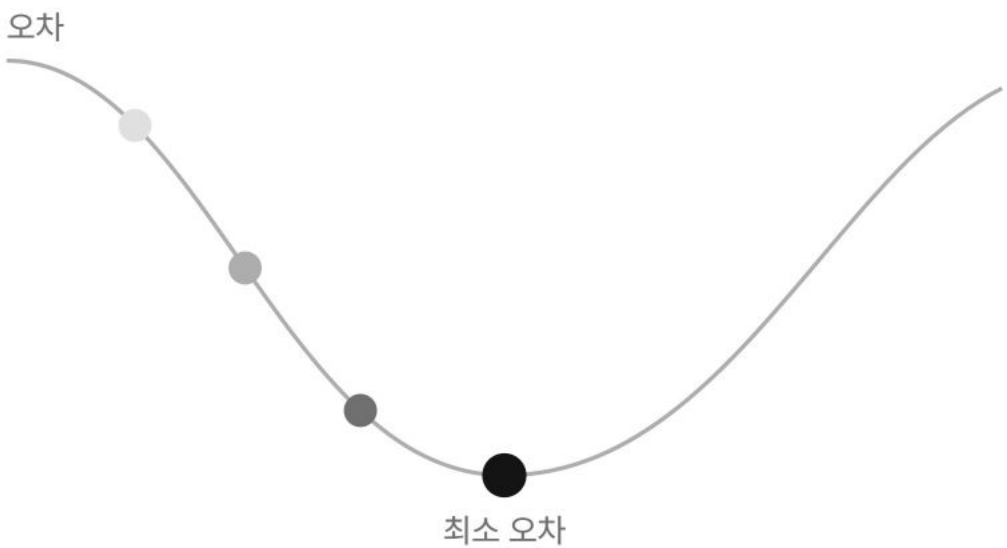
각 층의 Feature vector 수를 크게 줄여 전체 파라미터를 줄였습니다.

과적합 억제 →

Optimizer AdamW

Optimizer를 AdamW로 바꿔 Weight decay로 과적합을 억제했습니다.

Optimizer : 오차가 가장 작은 곳을 찾아 내려가는 방법



학습

모델은 가중치를 조금씩 바꾸며 오차가 줄어드는 방향으로 내려갑니다. 한 번 학습할 때 가중치를 바꾸는 크기가 Learning Rate(η)입니다.

Weight decay

가중치가 **지나치게 커지는 것을 억제**해 과적합을 완화합니다. 업데이트할 때마다 기존 가중치에 (1 − ηλ)를 곱해 조금씩 줄입니다.

$w \leftarrow (1 - \eta\lambda) \cdot w - \eta \cdot g$ · 아래 예: (1 − ηλ) = 0.9

1번째
2번째
3번째

AdamW

Adam에 Weight decay를 **기울기 업데이트와 분리해 따로** 적용한 Optimizer입니다.

04

CNN 개발의 정석

다양한 시도 끝에 다시 배운 개발 순서

다양한 시도와 성능의 부진

시도 1

Threshold 조정

판단 기준점을 바꿔도 구분 능력 자체는 그대로

시도 2

Memory Bank 도입

93장 · 74,400개 특징 → Accuracy 69.6%

시도 3

Kernel · 보간 · 증강

pool kernel 3→1
bilinear→nearest 데이터 증강

결과

Train 89% · Test 69%

학습 데이터에서만 잘 맞는 과적합 상태

결론적으로

처음부터 다시 공부

기법을 하나씩 붙여 보는 방식으로는 한계가 있었습니다. **신경망을 어떻게 학습시켜야 하는지**부터 다시 보기로 했습니다.

시도에 쓴 세 가지 기법

시도 2

Memory Bank

정상 사진에서 뽑은 특징(Feature vector)을 저장해 두는 창고입니다. 새 사진의 특징을 창고 속 가장 비슷한 특징과 비교해, **멀수록 이상**으로 봅니다.

새 사진 ↔ 거리 ↔ **정상 Bank**

시도 3

Pooling kernel 3 → 1

Pooling은 작은 틀(kernel) 안의 값을 하나로 요약하는 단계입니다. 틀을 3×3에서 1×1로 줄이면 주변과 섞이지 않아 **작은 흠이 덜 뭉개집니다**.

1	1	1
1	**9**	1
1	1	1

3×3 평균 → **1.9**
1×1 → **9** 그대로

시도 3

보간 bilinear → nearest

특징 지도를 크게 늘릴 때 빈 칸을 채우는 방법입니다. bilinear는 양옆 값을 섞어 부드럽게, nearest는 **가장 가까운 값을 그대로 복사**해 경계가 흐려지지 않습니다.

bilinear	0	3	6	9
nearest	0	0	9	9

“Backpropagation is a leaky abstraction.”

You can’t solve it with a few library calls; without understanding how it works, it is meaningless.

역전파는 새는 추상화다. 라이브러리 몇 줄로 해결되지 않으며, 원리를 이해해야 한다.

— Andrej Karpathy, “Yes you should understand backprop”

카파시의 방법을 따른 이유

01

데이터 품질 · 검증

데이터가 성능을 정한다

EDA를 하며 데이터 품질의 중요성을 알게 됐고, 모델보다 먼저 데이터의 정확도를 검증해야 한다는 것을 배웠습니다.

02

기법 활용

다양한 기법을 언제 쓸지 안다

TTA, 앙상블 같은 기법을 개발의 어느 단계에서 쓰면 좋은지 알 수 있었습니다.

03

개발 방식

정형화된 순서로 개발한다

기법을 하나씩 붙이는 대신, 따라갈 수 있는 정형화된 개발 방식이 필요했습니다.

CNN 개발 순서

Step 1

데이터를 눈으로 본다 (EDA)

몇 시간을 들여 데이터를 직접 보고 문제를 파악합니다. 필터, ROI, 불량의 특성이 여기서 정해집니다.

Step 2

큰 모델로 일부러 과적합

적은 데이터로 큰 모델을 돌려 100%에 수렴하는지 봅니다. 외우지도 못한다면 데이터 자체에 문제가 있는 것입니다.

마지막 단계

TTA · 앙상블 · 지식 증류

성능을 짜내는 기법은 앞 단계가 끝난 뒤에 도입합니다.

결과 → 이 글을 읽고, 데이터를 충분히 분석해 본 적이 없다는 것을 깨달았습니다. 그래서 1시간을 데이터 분석에만 썼습니다.

한 시간의 EDA가 바꾼 것

분석한 항목

데이터 크기 · 이미지 해상도 · **불량 원인 구분** · **찾기 어려운 불량** · 찾기 쉬운 불량 · **ROI가 필요한 부분** · 색 민감도 · **밝기 민감도**

적용한 것

ROI 추가	집중해야 할 영역을 지정
TTA 추가	찾기 어려운 불량에 대해 적용
밝기 필터	색은 문제마다 다른 의미를 가져 색 필터 대신 밝기 필터로 검증

어떤 문제가 심각하고, 무엇을 다시 분류해야 하는지 알 수 있었습니다.

4가지 적용 이론

ROI · Region of Interest

집중 구역

이미지에서 특정 부분에 집중하는 영역을 만듭니다.

TTA · Test Time Augmentation

다각도 추론

테스트 때 같은 이미지를 여러 형태로 바꿔 넣고 예측을 모아 성능을 높입니다.

MIL · Multiple Instance Learning

조각 모음 라벨링

이미지를 Patch로 쪼개 Bag에 넣고, Bag에만 라벨을 붙여 학습합니다.

Ensemble

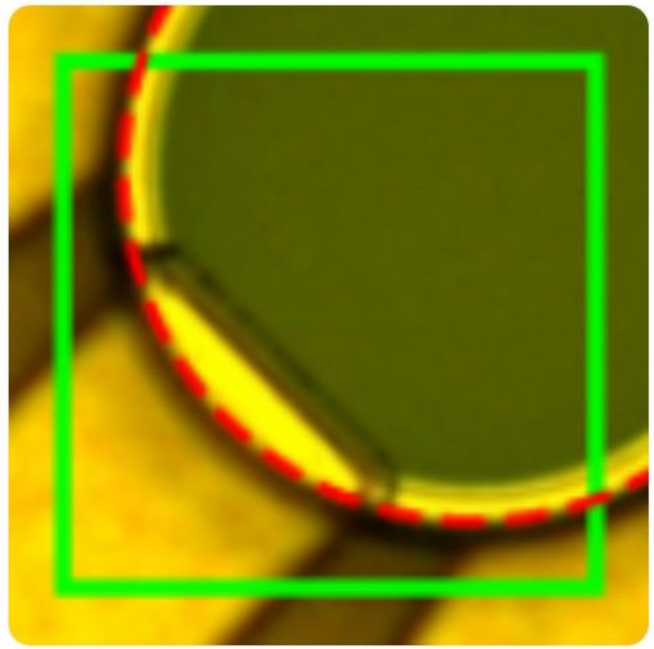
다중 모델

같은 모델을 서로 다른 데이터로 학습해 각각 추론한 뒤 예측값을 결합합니다.

ROI · 집중 구역



전체 사진 · 초록 상자 = ROI



ROI만 잘라 크게 본 모습

필요한 곳만 잘라 크게 본다

사진 전체를 넣으면 작은 불량이 넓은 배경에 묻힙니다. ROI는 자세히 봐야 하는 영역을 미리 정해, **그 부분만 잘라 모델에 따로 넣는 방법**입니다.

ROI는 정해진 위치라, 사진 속 다이오드의 위치가 바뀌면 다시 맞춰야 합니다.

TTA · 다각도 추론



원본



좌우 반전



상하 반전



90° 회전



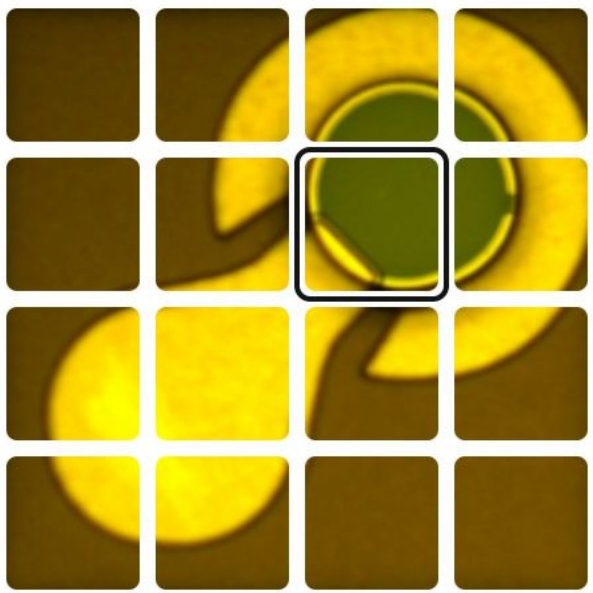
밝기 변화

사진 5장 → **같은 모델** → **점수 5개의 평균 = 최종 예측**

학습이 아니라 **테스트(추론) 때** 쓰는 기법입니다. 한 장면에서 우연히 틀리는 것을 줄여 줍니다.

MIL · 조각 모음 라벨링

Bag 라벨 · 불량



굵은 테두리 · 모델이 찾아낸 불량 조각 (예시)

01 **조각(Patch)으로 나누기** · 사진 한 장을 여러 조각으로 자릅니다.

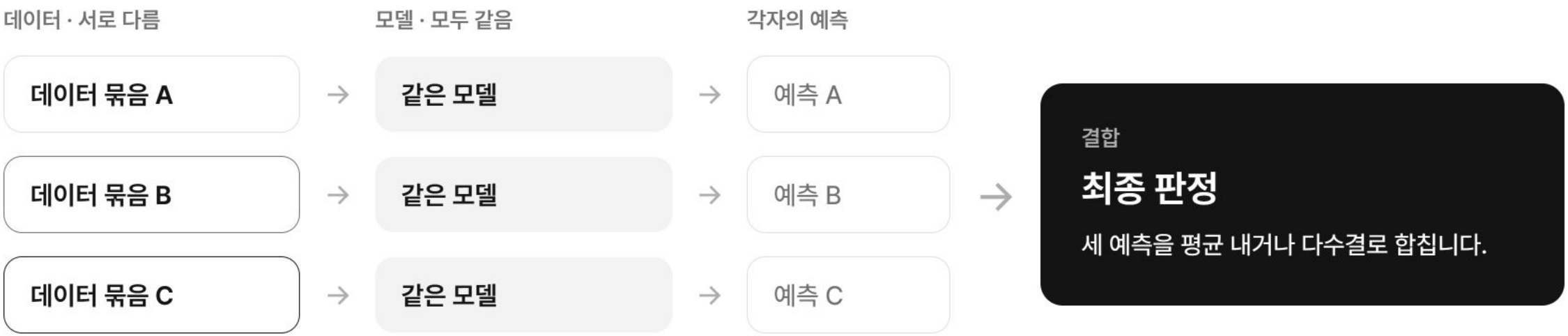
02 **Bag에만 라벨** · 조각을 모은 묶음이 Bag입니다. 라벨은 조각마다가 아니라 사진 단위로 하나만 붙입니다.

03 **하나라도 불량이면 불량** · 조각 중 하나라도 불량이면 Bag은 불량, 모두 정상이면 정상입니다.

04 **불량 조각은 모델이 찾기** · 어느 조각이 불량인지는 모델이 스스로 찾도록 학습합니다. 불량 위치를 일일이 표시할 필요가 없습니다.

• 용어 설명

Ensemble · 다중 모델



모델 구조는 같지만 **학습한 데이터가 달라** 판단이 조금씩 다릅니다. 여러 판단을 합치면 한 모델의 실수를 다른 모델이 보완합니다.

지식 증류 · Knowledge Distillation

Teacher

크고 정확한 모델

먼저 충분히 학습시킨 큰 모델

→

왜 쓰는가

여러 모델을 돌릴 환경이 안 될 때

앙상블처럼 여러 모델이나 큰 모델을 현장 장비에서 그대로 돌리기 어려울 때, **그 지식을 작은 모델 하나에 옮겨 담아** 씁니다.

→

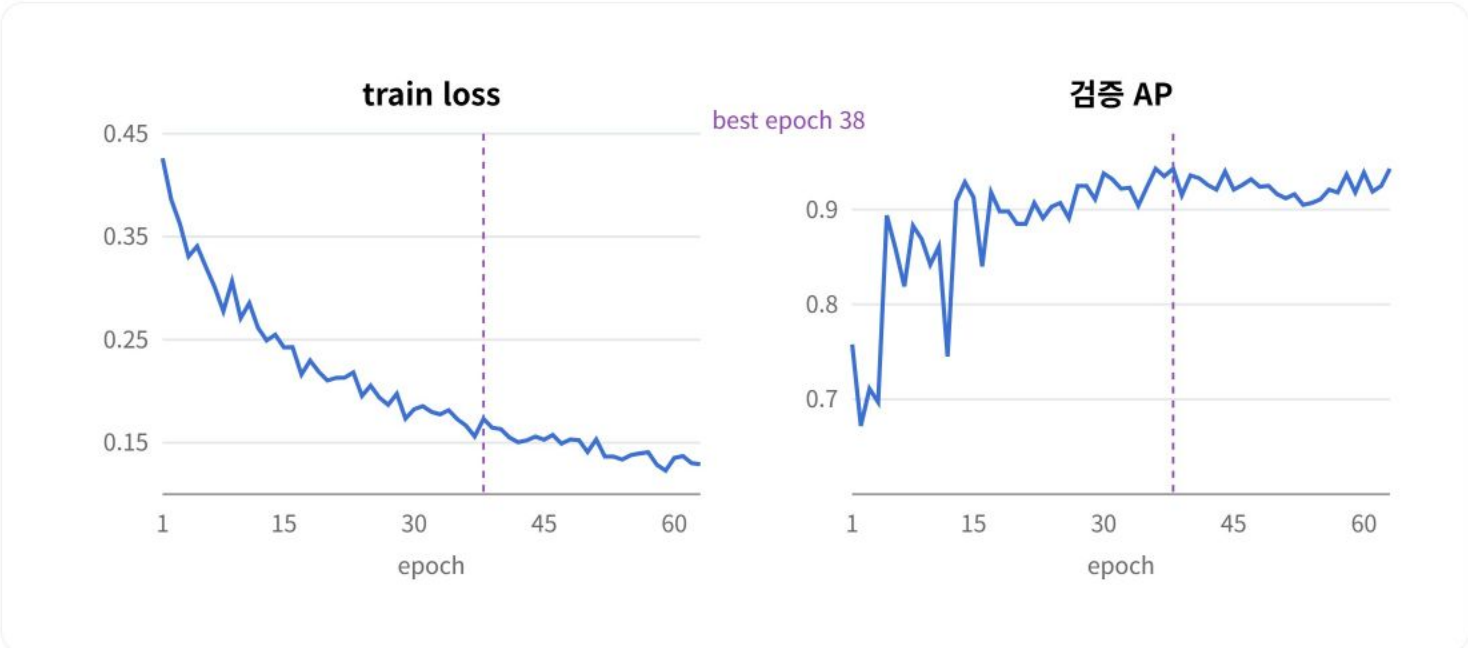
Student

작고 가벼운 모델

Teacher의 확률을 따라 배워, 작은 크기로 비슷한 성능을 노립니다.

카파시 개발 순서의 **마지막 단계** 기법으로, 앞 단계가 끝난 뒤 모델을 가볍게 만들 때 씁니다.

AP 0.8을 목표로 AUROC 평가



train loss · 검증 AP (best epoch 38)

데이터

4,300장가량

다시 라벨링해 Crop 이미지를 모아 학습했습니다.

교수님이 제시한 목표

AP 0.8 이상

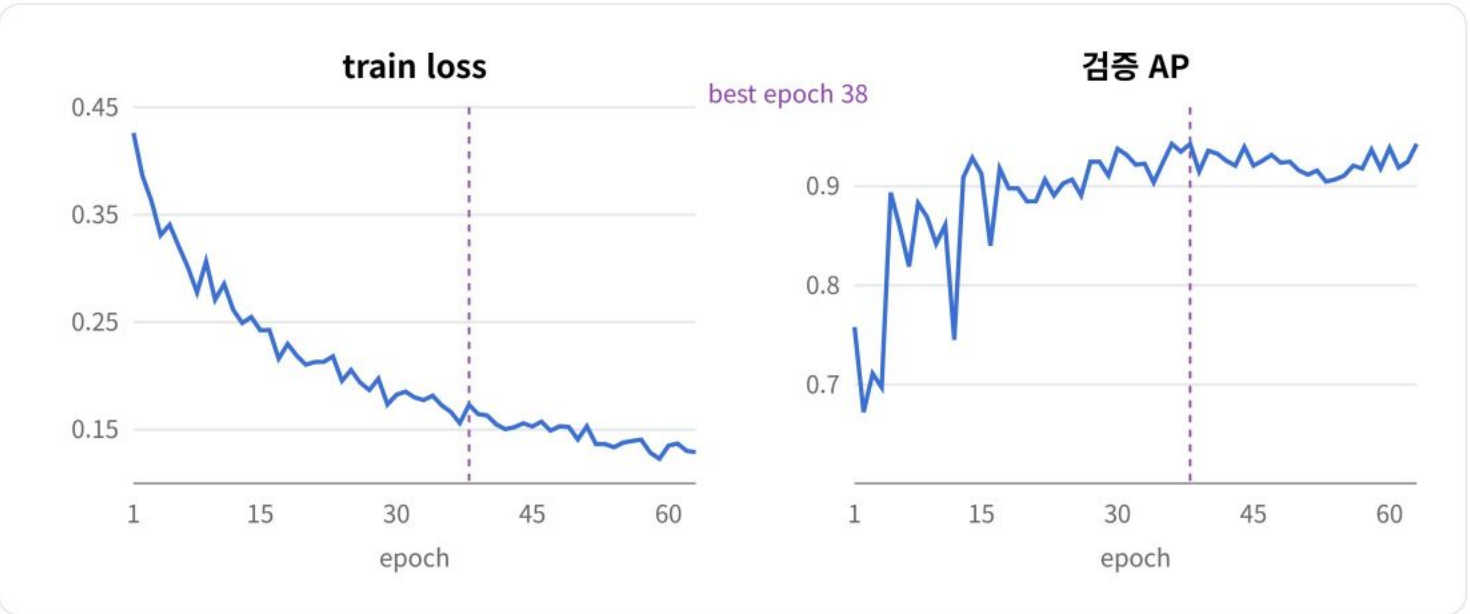
평가 지표는 AUROC. 경량화나 이론보다 개발에 집중했습니다.

결과 · 38 epoch

0.943

검증 AP로 목표치 0.8을 넘었습니다.

그래프로 본 학습 과정



train loss

0.43 → 0.13, 꾸준히 감소

학습 데이터는 epoch가 지날수록 더 잘 맞힙니다.

검증 AP

15 epoch 이후 0.9 근처에서 안정

초반에는 0.65~0.9 사이로 크게 흔들렸고, 38 epoch 이후로는 더 오르지 않았습니다.

best epoch 38

검증 AP가 가장 높은 지점을 저장

이후에도 loss는 내려갔지만 검증 AP는 오르지 않아, 38 epoch의 모델을 썼습니다.

그러나, 문제

우리가 한 것

데이터를 직접 다시 라벨링했다

분류 학습을 위해 라벨을 **우리 기준**으로 바꾸고, 그 데이터로 **AP 0.943**을 만들었습니다.

Phosem이 원한 것

정해진 라벨 그대로 평가받는 모델

회사는 라벨을 임의로 바꾸는 것을 원하지 않았습니다. **현장에서 내린 판단**을 그대로 믿고, 그 기준대로 분류하는 모델이 필요했습니다.

우리가 라벨링한 4,300장

0.943 → 0.69

누구도 건드리지 않은 새 데이터 3,609장 · Test dataset

새 카메라로 찍은 3,609장으로 재학습한 결과, Test dataset에서 AP 0.69의 모델이 나왔습니다.

데이터 차이



기존 데이터



새 카메라 데이터 (3,609장 중 일부)

빛
조명 밝기가 제각각

색
배경 색이 크게 달라짐

위치
다이오드가 잘린 위치가 다름

→

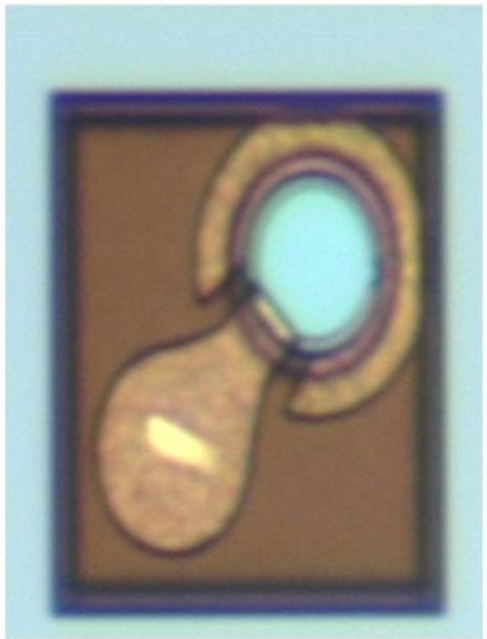
Filter
Filter 조건이 달라짐
원본 데이터에 맞춘 Filter 조건이 그대로 통하지 않음

→

ROI
기존 ROI가 맞지 않음

PIN 다이오드 불량 검출 회고

R1~R4: 촬영 회차



R1 · 1차 촬영



R2 · 2차 촬영



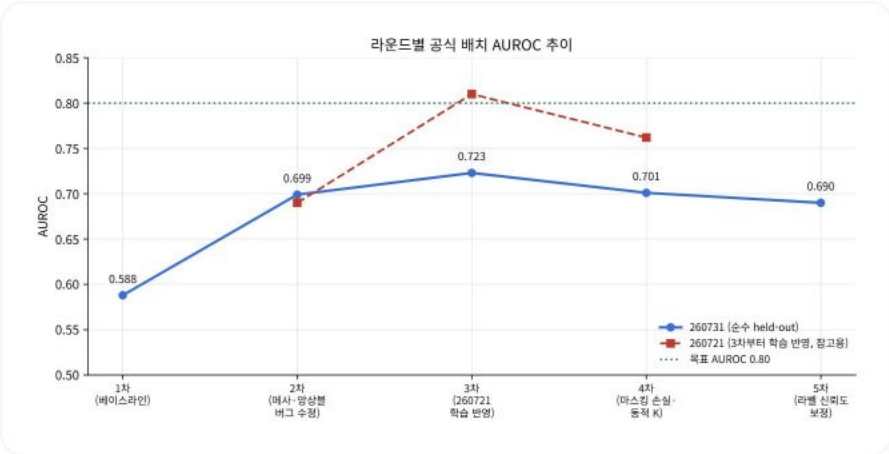
R3 · 3차 촬영



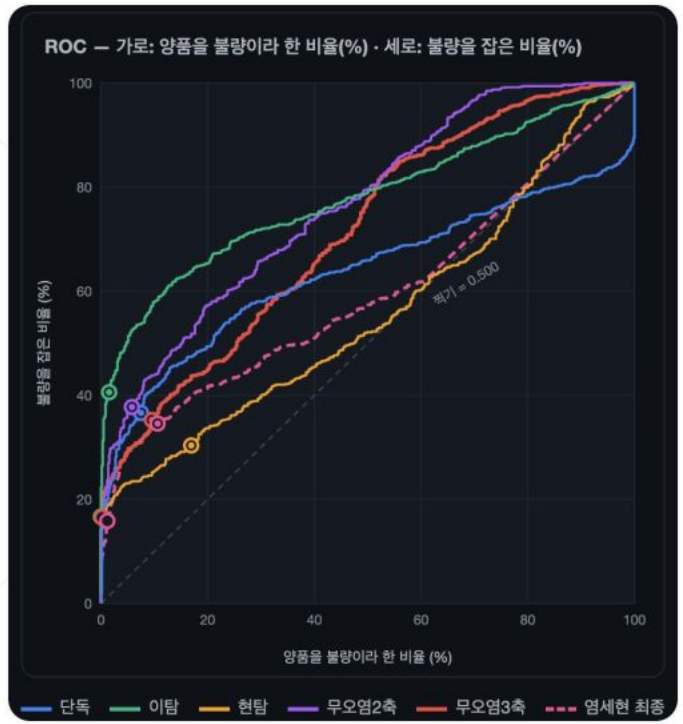
R4 · 4차 촬영

다이오드는 최대 4번 촬영합니다. 첫 촬영에서 쓸 만한 사진이 나오면 R1, 아니면 R2·R3로 넘어가고, **가장 구분이 어려운 다이오드가 R4**가 됩니다.
같은 다이오드라도 회차마다 색과 초점이 달라집니다.

다른 모델보다 부족했던 성능



3,609장 학습·채점 성능 추이



ROC — 대각선에 가까울수록 찍기와 같음

내 모델 · 미라벨 전체 성능 채점

0.59

AUROC

성능 추이 그래프는 3,609장으로 학습·채점한 결과를 보여 줍니다. ROC 그래프에서 다른 모델에 비해 내 모델의 곡선이 대각선 가까이에 머물러 있습니다.

05

새로운 모델

교수님의 제안으로 개발된 새 모델

• 용어 설명

이상 탐지 방식과 PatchCore

방식	정상을 기억하는 방법	이상 판정	추가 학습
Autoencoder	정상 사진을 작게 압축했다가 원래대로 복원하도록 학습	복원한 사진과 원본의 차이가 크면 이상	필요
PaDiM	사진의 칸마다 정상 특징의 평균과 흔들리는 범위를 저장	같은 칸의 정상 범위에서 많이 벗어나면 이상	없음 · 위치가 정렬돼야 함
Normalizing Flow	정상 특징이 어떤 값으로 자주 나오는지 학습	정상에서 거의 나오지 않는 값이면 이상	필요
PatchCore	정상 사진의 조각 특징을 그대로 저장 (대표 조각만 남김)	가장 비슷한 정상 조각과도 거리가 멀면 이상	**없음** · 위치와 상관없이 비교

네 방식 모두 정상 데이터만으로 만듭니다. PatchCore는 이미 학습된 ResNet50의 특징을 그대로 쓰기 때문에 **새로 학습시킬 필요가 없고**, 저장해 둔 정상 조각이 곧 판정 기준입니다.

Anomaly Detection

지금까지 · 분류

불량의 종류를 가르친다

불량 원인별로 충분한 예시가 있어야 합니다. 스크래치 65장으로는 부족했습니다.

이상 탐지 · Anomaly Detection

정상의 모습만 기억한다

정상 데이터의 특징(Bank)과 새 데이터의 특징을 비교해, 많이 다르면 이상으로 봅니다. 양품이 많은 이 데이터에 맞는 방식입니다.

Feature vector	데이터에서 추출한 특징 벡터
Outlier	한 데이터셋 안에서 서로 비교해 튀는 데이터를 찾습니다
Novelty	한 데이터셋으로 Bank를 만들고, 새로운 데이터의 이상을 탐지합니다

메모리 뱅크와 PatchCore

① 나누기

정상 사진을 조각으로

ResNet50이 사진을 격자 조각으로 나누고, 조각마다 Feature vector를 뽑습니다.

→

② 저장

대표 조각만 Bank에

비슷한 조각은 하나만 남기는 Coreset으로, 적은 수로 정상 전체를 덮습니다.

→

③ 비교

가장 먼 조각 = 이상 점수

새 사진의 조각마다 Bank에서 가장 가까운 정상 조각과의 거리를 잽니다. 그중 가장 큰 값이 사진의 이상 점수입니다.

핵심

왜 조각인가

불량은 사진 전체가 아니라 **작은 한 부분**에 생깁니다. 사진 한 장을 숫자 하나로 요약하면 작은 흠이 묻히지만, 조각 단위로 비교하면 **어느 부분이 다른지** 드러납니다.

신뢰 가능한 정상 Bank를 위한 Pipeline

1

입력

MIL 상위 900장을 뺀 2,142장

MIL이 불량으로 본 상위 900장에 들지 않은 데이터에서 **Feature vector**를 추출합니다.

ResNet50 Layer 2·3 사용

→

2

Outlier

한 장씩 자기검증

한 장을 나머지와 비교해 (본인 제외) **튀는 데이터를 제외**합니다.

데이터셋 내에서 검증

→

3

Novelty

정상 Bank 확보

정렬 표식 사진을 제외한 데이터로 **정상 Bank**를 만듭니다.

Coreset 214만 → 10만 개

→

4

판정

PatchCore

새 다이오드의 조각을 Bank와 비교해, **거리가 멀면 불량**으로 판정합니다.

정상/불량 라벨 없이 동작

PIN 다이오드 불량 검출 회고

ResNet50의 Layer별 특징

ResNet50은 이미지에서 특징을 추출하는 CNN 모델입니다. 신경망은 여러 레이어를 거치면서 이미지의 특징을 단계적으로 추출합니다.

일반적으로 뒤쪽 레이어로 갈수록 더 추상적이고 의미적인 특징을 학습하는 경향이 있습니다.

Layer 1

단순한 경계, 모서리, 기초적인 질감

Layer 2 사용

비교적 국소적인 형태와 질감

Layer 3 사용

더 넓은 영역의 형태와 복합적인 특징

Layer 4

더 추상적이고 의미적인 특징

단순 ——— 추상·의미적

Layer 2·3을 고른 이유 · 불량은 작은 형태·질감의 차이라, 너무 단순한 Layer 1과 너무 추상적인 Layer 4 대신 그 사이의 Layer 2·3을 썼습니다.

New Feature vector

입력 사진

2,142장

868×868로 맞춘 정상 다이오드

→

후보 조각

2,142,000

109×109 격자에서 한 장당 1,000개씩 뽑은 조각 특징

→

Coreset · 정상 Bank

100,000

전체를 대표하는 조각만 선택 (4.7%)

```
정상 풀: 2,142장 / 패턴 밖 제외: ['R1_-18_21 (1).PNG'] / 정제 제외: 0장 / 뱅크 사용: 2,142장
입력: 868×868 / 특징맵: 109×109 = 11,881조각 / 장당 표본: 1,000 / 후보 전체: 2,142,000
특징 추출: 100% 134/134 [01:42<00:00, 1.38it/s]
특징 추출 완료: 103초
코어셋 선택: 100% 100000/100000 [12:26<00:00, 122.36it/s]
coreset 선택 완료: 747초
메모리 뱅크: (100000, 1024) / 중복 없는 선택: 100,000
샘플링 비율(후보 대비): 4.7% / 커버 반경(투영 공간): 7.0770
사진당 뱅크 조각: 최소 9 / 중앙 37 / 최대 436 / 0개인 사진 0장
저장: /content/drive/MyDrive/300_data/banks/bank_868_coreset_ex0.pt
```

실제 Bank 생성 로그 · 메모리 뱅크 (100,000, 1024)

Coreset을 고르는 방법

가장 멀리 떨어진 조각을 하나씩 추가하는 과정을 10만 번 반복해, **적은 수로 전체를 덮는** 조각 묶음을 만듭니다.

최대 거리 7.0770

버린 조각마다 가장 가까운 남긴 조각까지의 거리 중 최댓값입니다.

(100,000, 1024)의 뜻

조각 번호	열 1	열 2	열 3	…	열 1024
1	0.2	1.3	0.0	…	0.8
2	0.1	0.9	0.4	…	0.7
…	…	…	…	…	…
100,000	0.5	0.0	2.1	…	0.3

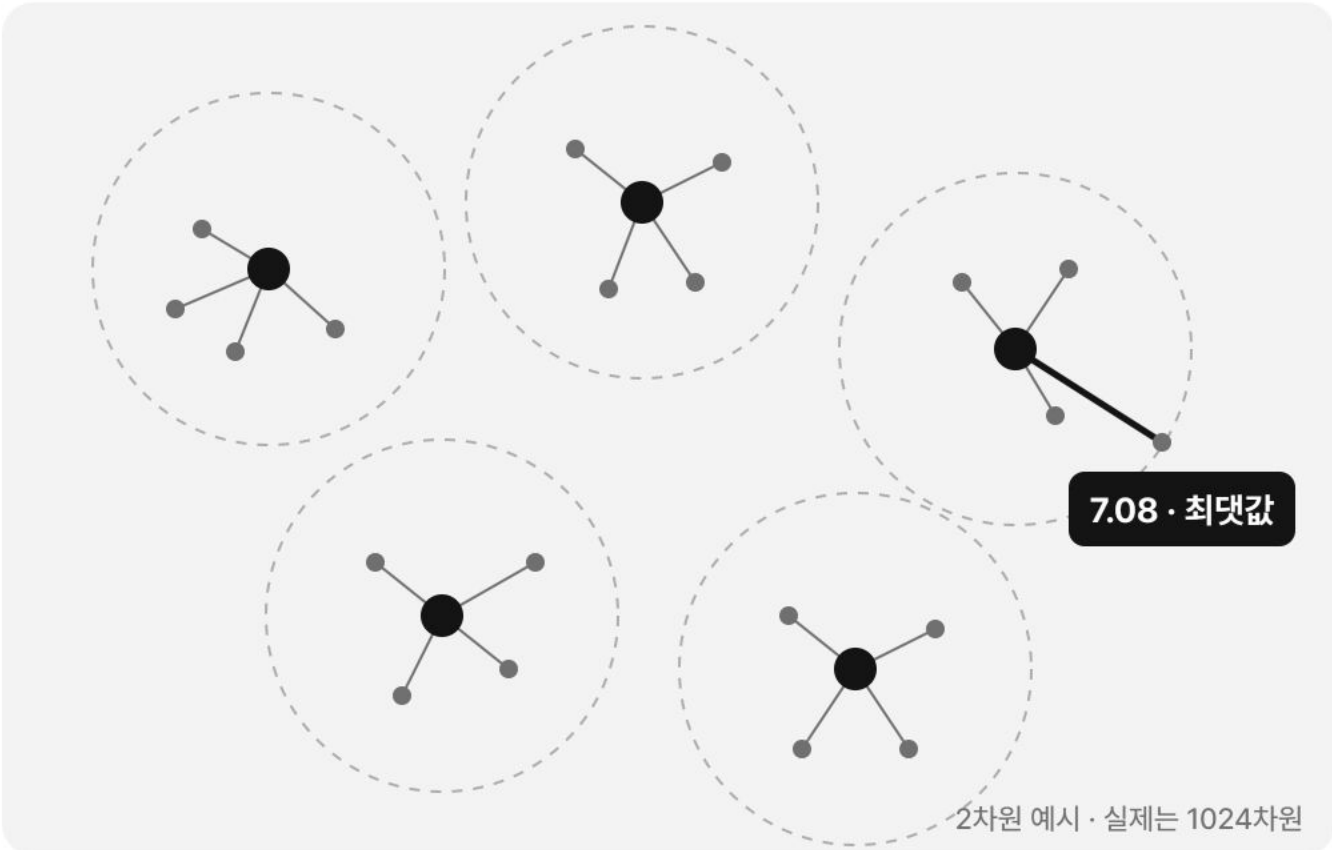
100,000 **행** · 조각의 개수 (Coreset)

1024 **열** · 조각 하나를 설명하는 숫자의 개수. ResNet50 **layer3의 채널 수(1024)**와 같습니다.

7.08 **거리** · 1024개 숫자로 측정한 특징 간 거리입니다.

조각 하나는 숫자 1024개를 포함하는 배열(**특징 벡터**)입니다. 숫자 하나가 아니라 배열 전체가 사진 한 칸의 특징이고, 거리는 이 배열 두 개를 비교해 구합니다.

“버린 조각은 모두 7.08 안에 있다”의 뜻



● **남긴 조각** · Bank 10만 개

● **버린 조각** · 약 204만 개

—— 버린 조각 → 가장 가까운 남긴 조각

····· 반지름 7.08인 원

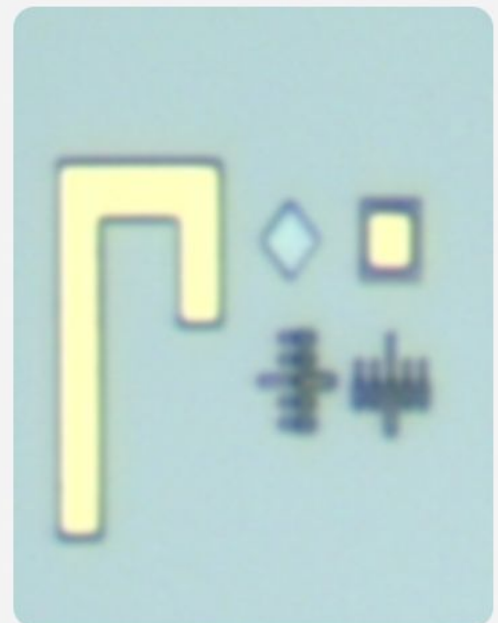
① 버린 조각마다 가장 가까운 남긴 조각을 찾아 선으로 잇습니다.

② 그 선 중 **가장 긴 것이 7.08**입니다. 대부분의 선은 이보다 짧습니다.

③ 그래서 모든 버린 조각은 어느 남긴 조각의 **원 안**에 들어갑니다. 평균이 아니라 최악의 경우 기준입니다.

가장 먼 특징

Bank와 거리가 가장 먼 특징을 확인해 보니, R4 사진에 섞인 정렬 표식이었습니다. 이 사진들을 걸러낸 뒤 Feature vector를 다시 뽑아 새 Bank를 만들었습니다.

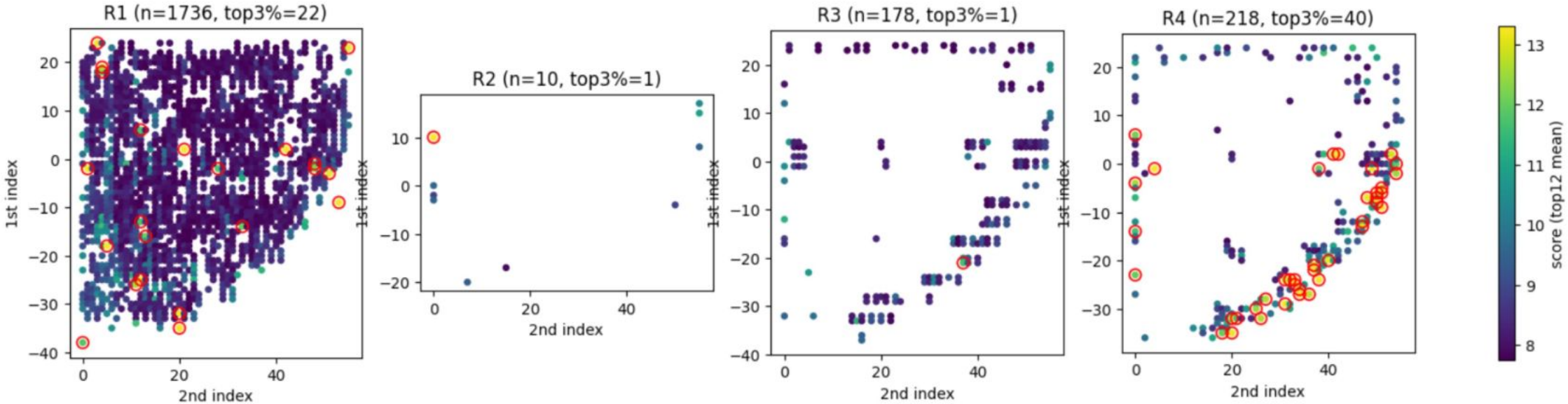


정렬 표식

촬영 위치를 맞추는 표시

다이오드가 아닌 표식이 사진에 함께 찍혀 이상으로 잡혔습니다.

회차별 이상 점수 그래프 읽는 법



칸 4개
R1~R4 촬영 회차별로 나눈 그래프

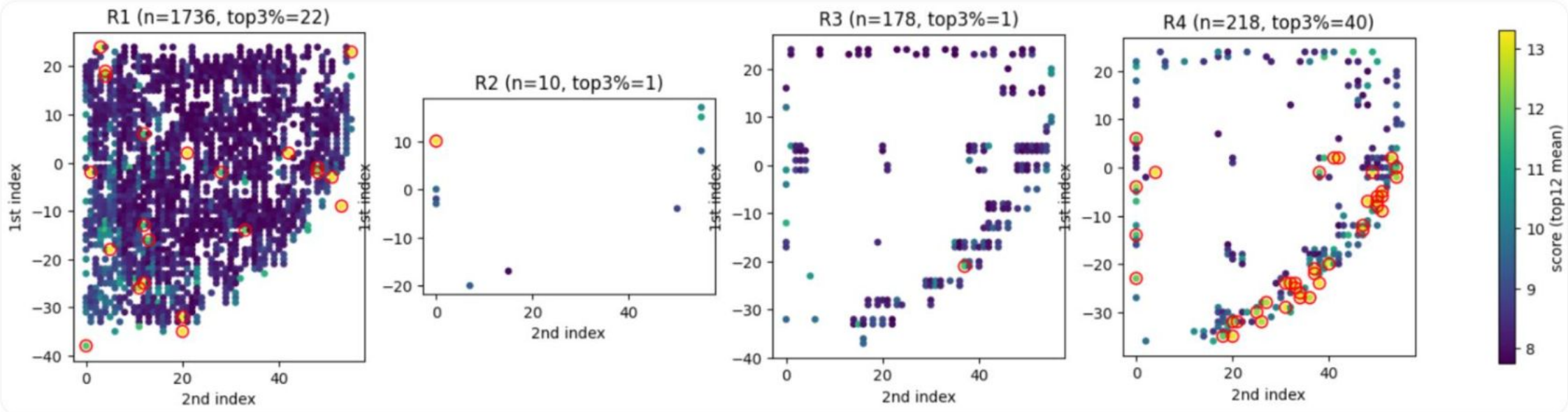
점 하나
다이오드 하나를 의미하며, 데이터 파일에 적힌 다이오드 위치를 기반으로 점 배치

색
이상 점수 높음 : 노랑
이상 점수 낮음 : 보라

빨간 원
전체에서 이상 점수 상위 3%

제목 n / top3%
회차의 다이오드 수 / 그중 상위 3%에 든 수

결과에서 R4가 유독 이상해 보였던 이유



촬영 회차(R1~R4)별 이상 점수 분포 — 빨간 원은 상위 3%

다이오드는 최대 4번 촬영합니다. 첫 촬영에서 쓸 만한 사진이 나오면 R1, 아니면 R2·R3로 넘어가고, 가장 구분이 어려운 다이오드가 R4가 됩니다. 그래서 **R4의 이상 점수가 높은 것은 모델 오류가 아니라 예상된 결과**였습니다. 기존 MIL 모델에서도 R4의 불량 확률이 높았습니다.

• 용어 설명

Confusion matrix · 판정을 네 칸으로 나누기

	불량이라고 예측	정상이라고 예측
실제 불량 100개	**60** TP · 맞게 잡음	**40** FN · 놓침
실제 정상 100개	**20** FP · 잘못 잡음	**80** TN · 맞게 통과

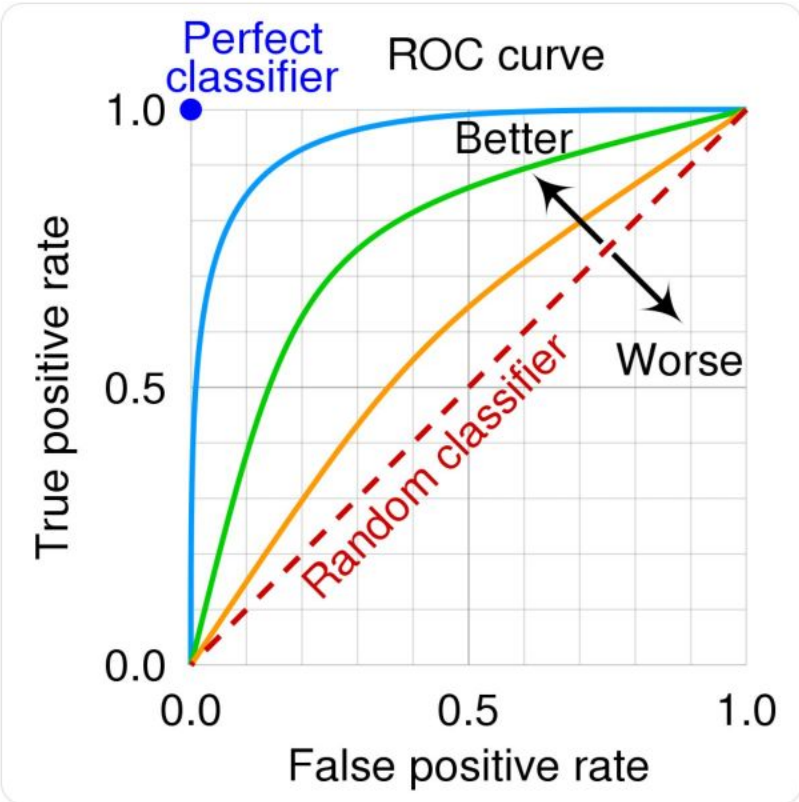
TPR = Recall — **0.6**
실제 불량 중 잡은 비율 · 60 ÷ 100

FPR — **0.2**
실제 정상 중 잘못 잡은 비율 · 20 ÷ 100

Precision — **0.75**
불량이라고 한 것 중 진짜 불량 · 60 ÷ (60 + 20)

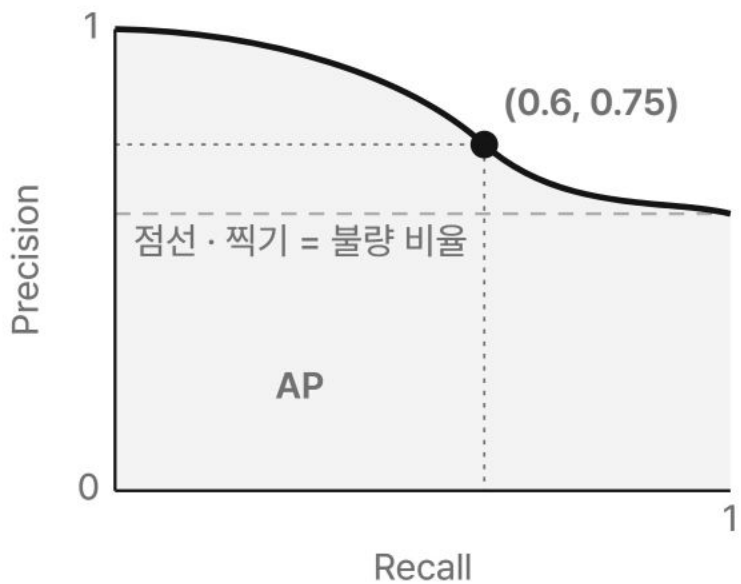
ROC는 FPR·TPR, **PR**은 Recall·Precision을 축으로 씁니다.

ROC curve 읽는 방법

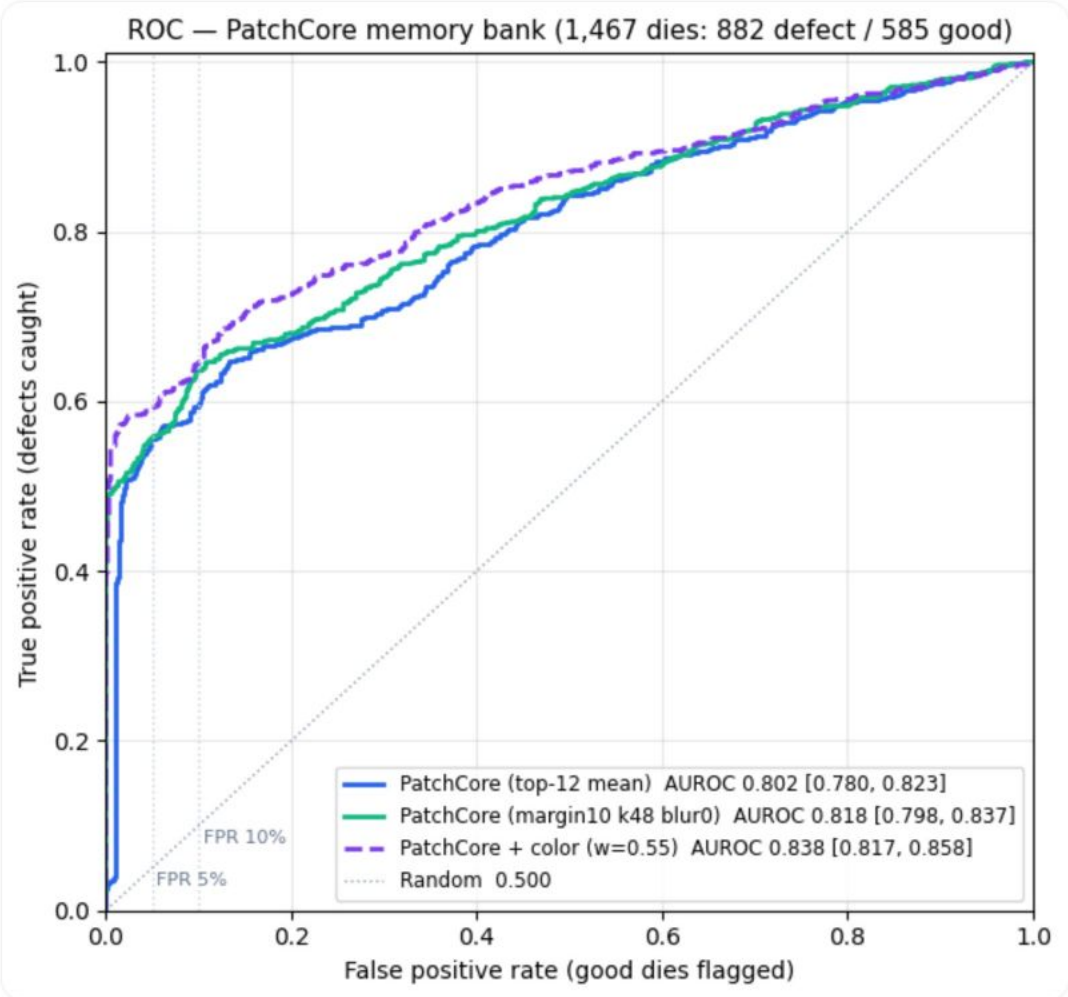


세로축 **True positive rate**	실제 불량 중 **불량이라고 맞게 잡은 비율**. TP ÷ 실제 불량 수. 높을수록 좋습니다.
가로축 **False positive rate**	실제 정상 중 **불량이라고 잘못 잡은 비율**. FP ÷ 실제 정상 수. 낮을수록 좋습니다.
곡선 **점들을 잇기**	모델은 데이터를 가지고 불량을 예측하고, 점수를 산출합니다. 이 점수를 기준으로 Threshold를 바꿀 때마다 (가로, 세로) 점이 하나 생깁니다.
면적 **AUROC**	곡선 아래 면적. **왼쪽 위에 붙을수록 1**에 가깝습니다. 즉, Perfect classifier에 가까울수록 좋은 모델입니다.

PR curve와 AP 읽는 방법



가로축 **Recall**	실제 불량 중 **잡아낸 비율**. ROC의 세로축과 같습니다.
세로축 **Precision**	불량이라고 한 것 중 **실제 불량의 비율**. 높을수록 판정을 믿을 수 있습니다.
곡선 **점들을 잇기**	예: 불량 100개 중 60개를 잡고, 불량이라고 한 80개 중 60개가 진짜면 → (0.6, 0.75)
면적 **AP**	곡선 아래 면적. **오른쪽 위에 붙을수록 1**에 가깝고, 점선은 찍기 수준인 데이터 속 불량 비율입니다.



다이오드 1,467개 · 불량 882 / 양품 585

PatchCore 단독 AUROC 0.802

0.802

Filter ① · top-12 mean

거리가 가장 먼 조각 12개의 평균을 이상 점수로 사용

0.818

Filter ② · margin10 · k48 · blur0

가장자리 10 제외 · k = 48 · 블러 없음

0.838

Filter ③ · + color (w = 0.55)

색 정보 점수를 비중 0.55로 더함

색 정보를 더해 오른 0.036(0.802 → 0.838)이 **결함을 본 것인지, 촬영 조건의 색 차이를 본 것인지는 확인하지 못했습니다.**

모델 조합

A

PatchCore만

PatchCore 단독 · AUROC 0.802

B

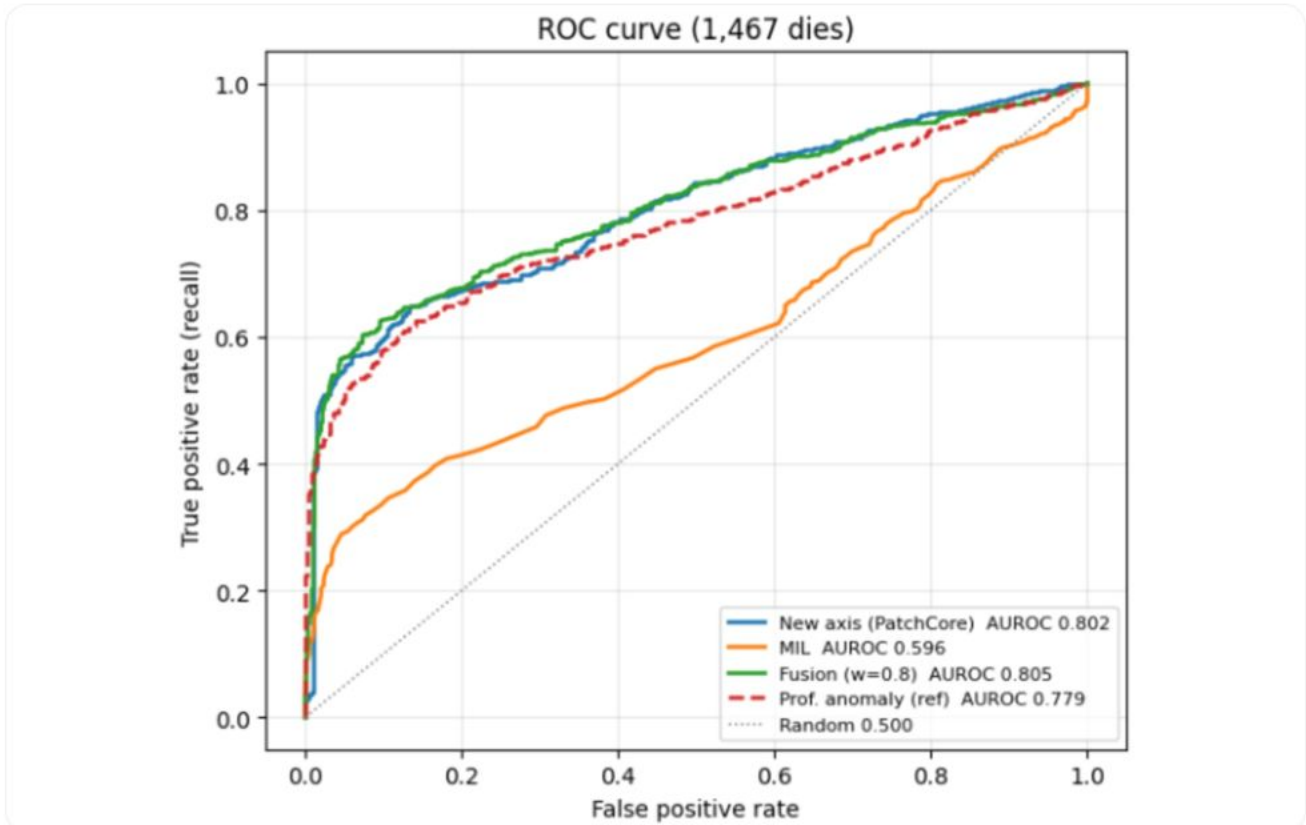
MIL만

기존 모델 단독 · AUROC 0.596

C 채택

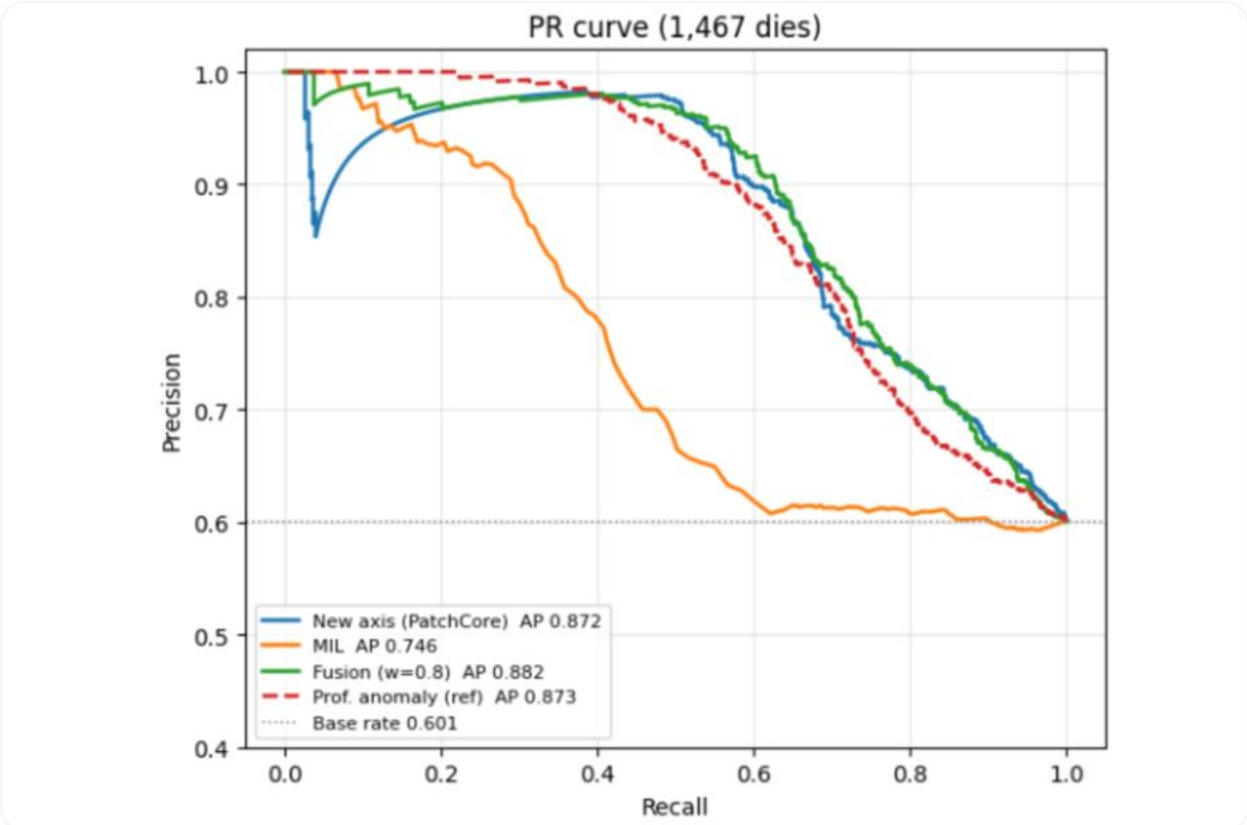
두 모델을 합친다

Fusion (w = 0.8) · AUROC 0.805



ROC curve · 1,467장 기준 — 왼쪽 위로 붙을수록 좋음

PR curve와 AP로 본 성능



PR curve · 1,467장 기준 — 오른쪽 위로 붙을수록 좋음

PR curve

가로는 Recall(실제 불량 중 잡아낸 비율), 세로는 Precision(불량이라고 한 것 중 실제 불량 비율)입니다. ROC와 달리 **불량 판정이 얼마나 믿을 만한지**를 보여 줍니다.

AP

PR curve 아래 면적. 점선(Base rate 0.601)은 찍기 수준입니다.
- Base rate : 데이터 속 불량 비율(882 ÷ 1,467). 무작위로 판단하면 AP가 이 값이 됩니다.

MIL	0.746
PatchCore	0.872
Fusion (w = 0.8)	**0.882**

MIL은 Recall이 오를수록 Precision이 빠르게 떨어지지만, PatchCore와 Fusion은 높은 Precision을 더 오래 유지합니다.

PNN 머티리얼즈 불량 검출 리포

Spearman 상관계수와 결합 비율 w

Spearman 상관계수

두 모델이 매긴 순위가 얼마나 비슷한가

−1~1 사이 값. **1이면 순위가 똑같고, 0에 가까우면 서로 다르게 판단**합니다. 서로 다른 단위로 판별하는 두 모델이 매긴 순위를 가지고 판단합니다.

다이오드	A	B	C	D	E
MIL 순위	1	2	3	4	5
PatchCore 순위	4	1	5	2	3

예시 · 순위가 서로 엇갈리면 값이 0에 가까워집니다.

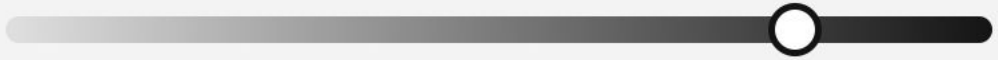
0.173 실제 값 · 두 모델이 **거의 다르게 판단** → 합치면 보완할 여지

결합 비율 w

두 순위를 섞는 비중

두 모델의 점수를 순위로 바꾼 뒤, w만큼 PatchCore를, 나머지만큼 MIL을 반영합니다.

결합 = w × PatchCore + (1 − w) × MIL



w = 0 · MIL만 w = 1 · PatchCore만

0.8 w를 0부터 1까지 바꿔 보며 찾은 값 · **PatchCore 80% + MIL 20%**

MIL : PatchCore = 2 : 8

먼저 Spearman 상관계수(0.173)로 두 모델이 서로 다르게 판단한다는 것을 확인했습니다. 그다음 두 점수를 순위로 바꿔 비율 w로 합치고, w를 0부터 1까지 바꿔 가며 가장 좋은 지점을 찾았습니다.

Phosem은 이렇게 개발된 PatchCore + MIL 모델을 사용하기로 결정했습니다.

w별 결합 성능 (w = 새 축 비중)

w	AUROC	AP
0.0	0.5965	0.7457
0.1	0.6203	0.7665
0.2	0.6484	0.7845
0.3	0.6811	0.8035
0.4	0.7207	0.8259
0.5	0.7601	0.8501
0.6	0.7860	0.8691
0.7	0.8001	0.8801
0.8	0.8051	0.8824
0.9	0.8050	0.8783
1.0	0.8020	0.8724

w별 결합 성능 (w = PatchCore 비중)

06

회고와 배운 점

걸림돌, 해결책, 처음 해 본 것, 그리고 다음

네 달의 흐름

2026 · 07

CNN 분류 시작

250장, ResNet. 과적합과 데이터 부족을 처음 마주함

2026 · 08

멘토링 · 피드백

모델 경량화, AdamW, 재라벨링

2026 · 09

새 데이터 3,609장

직접 라벨링한 데이터의 0.943이 새 데이터에서 0.69로

2026 · 10

PatchCore + MIL 완성

PatchCore + MIL Fusion, Phosem 채택

신경망 설계에서 중요했던 것

걸림돌

기본기 없이 AI에 판단을 맡겼다

무엇이 중요한지 모르니 기술 선택의 근거가 없었고, 레퍼런스가 많은 모델을 무작정 골랐습니다.

해결책

데이터에 맞는 문제로 바꿨다

불량 예시가 적다면 정상만 배우는 이상 탐지로. 정상 Bank도 스스로 검증해 만들었습니다.

처음 해 본 것

데이터를 오래, 직접 봤다

EDA로 ROI·TTA·밝기 필터를 정하고, R4와 정렬 표식처럼 숫자 뒤의 원인을 찾았습니다.

효과를 본 것

결과로 비교하고 골랐다

같은 1,467장 기준표로 세 가지 안을 비교해, AP 0.882의 Fusion을 근거와 함께 제안했습니다.

바랐던 환경과 실제 현장

바랐던 것	현장의 현실
불량 원인별로 충분한 샘플	양품 : 불량 = 6 : 1, 스크래치는 65장
정확하고 일관된 라벨	도중에 재라벨링, 촬영 회차(R1~R4)마다 다른 난이도
우리가 정리한 데이터로 평가	누구도 건드리지 않은 3,609장으로 채점과 AP 하락

현장에서도 이 데이터로 구분할 수 있어야 하기 때문에 개발은 그대로 진행되었습니다. 이상적인 조건을 기다리는 대신 주어진 데이터로 답을 내야 했고, 처음 겪어 본 꽤 어려운 개발이었습니다.

도달한 지점과 다음 과제

도달한 지점

0.805

Fusion AUROC · 1,467장 기준

0.882

Fusion AP · Phosem 채택

기회가 된다면

- R4 데이터를 따로 모아 별도로 학습
- PatchCore가 틀린 데이터를 EDA해 알맞은 Filter를 추가
- Filter로 불필요한 정보를 걸러 더 좋은 정상 Bank 확보

이 일을 이어받는 분께

01 · Look at the data

데이터부터 보세요

모델을 고르기 전에 몇 시간이라도 사진을 직접 보세요. 정렬 표식, 촬영 회차(R1~R4)가 점수를 흔듭니다.

02 · Trust the bank

정상 Bank를 의심하세요

Bank의 품질이 PatchCore의 성능입니다. 자기검증(Outlier) 단계로 정상 데이터셋을 계속 정제하세요.

03 · Score on new data

새 데이터로 채점하세요

직접 라벨링한 데이터의 숫자는 높게 나옵니다. 아무도 건드리지 않은 데이터로 비교해야 합니다.

04 · Study the basics

기본기 공부를 먼저 하세요

무엇이 중요한지 스스로 판단하려면 신경망의 기본 원리를 충분히 공부하는 것이 좋습니다.

05 · Next step

BBox 라벨링 → YOLO26 + PatchCore

BBox 라벨링 → YOLO26 학습 → PatchCore와 결합

YOLO 모델은 일반 CNN보다 불량 위치와 원인을 더 잘 잡을 것으로 기대합니다(검증 전). BBox로 라벨링해 YOLO26을 학습하고, PatchCore와 합쳐 볼 수 있습니다.

PhoSem
Photonic.Semiconductor

감사합니다

정답에 가까운 방법을 배우고, 그 방법을 스스로
선택하며 발전하는 것이 중요하다는 것을 배웠습니다.